

Wykład 6

Modele języka naturalnego. Przetwarzanie języka naturalnego

NLP-natural language processing

Transformery

NLP natural language processing

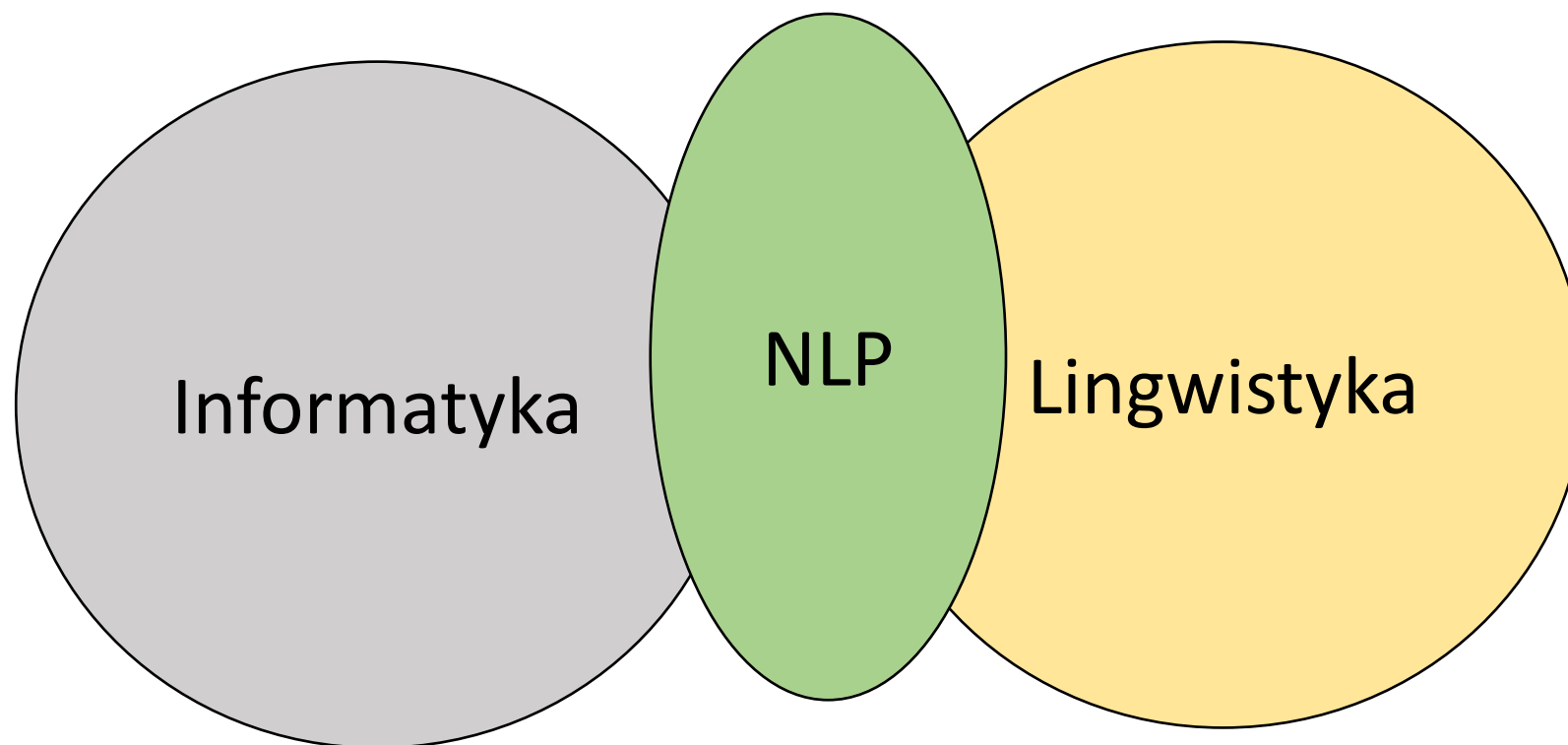
- Przetwarzanie języka naturalnego (NLP) to aspekt sztucznej inteligencji, który pomaga komputerom rozumieć, interpretować i wykorzystywać ludzkie języki.
- NLP pozwala komputerom komunikować się z ludźmi, używając ludzkiego języka. Przetwarzanie języka naturalnego zapewnia również komputerom możliwość czytania tekstu, słuchania mowy i interpretowania jej.
- NLP czerpie z kilku dyscyplin, w tym lingwistyki obliczeniowej i informatyki, próbując wypełnić lukę między komunikacją ludzką a komputerową.

NLP natural language processing

- NLP wykorzystuje zaawansowane technologie, takie jak lingwistyka komputerowa czy zaawansowane modele uczenia maszynowego i głębokiego uczenia, aby umożliwić komputerom przetwarzanie ludzkiego języka – zarówno w formie tekstu, jak i innych form, takich jak nagrania głosowe.
- Głównym celem rozwiązań wykorzystujących NLP jest nie tylko zrozumienie znaczenia poszczególnych słów, ale całej wypowiedzi, uwzględniając przy tym kontekst, intencje, a nawet emocje osoby, której wypowiedź jest przetwarzana.

Definicja (Wikipedia)

Przetwarzanie języka naturalnego (ang. natural language processing, NLP) – interdyscyplinarna dziedzina, łącząca zagadnienia sztucznej inteligencji i językoznawstwa, zajmująca się automatyzacją analizy, rozumienia, tłumaczenia i generowania języka naturalnego przez komputer. System generujący język naturalny przekształca informacje zapisane w bazie danych komputera na język łatwy do odczytania i zrozumienia przez człowieka. Zaś system rozumiejący język naturalny przekształca próbki języka naturalnego na bardziej formalne symbole, łatwiejsze do przetworzenia dla programów komputerowych. Wiele problemów NLP wiąże się zarówno z generacją, jak i rozumieniem języka, np. model morfologiczny zdania (struktura słów), który komputer powinien zbudować, jest potrzebny zarazem do tego, by zdanie było zrozumiałe, jak i gramatycznie poprawne.



Elementy historii

- Podwaliny położył szwajcarski profesor lingwistyki Ferdinand de Saussure (1857-1913), znany z koncepcji „języka jako nauki”, która ostatecznie doprowadziła do przetwarzania języka naturalnego.
- W latach 1906-1911 profesor Saussure prowadził trzy kursy na Uniwersytecie Genewskim, gdzie opracował podejście opisujące języki jako „systemy”. W ramach języka dźwięk reprezentuje pojęcie, które zmienia znaczenie wraz ze zmianą kontekstu.

Elementy historii

- W 1950 roku Alan Turing napisał artykuł opisujący test na „myślącą” maszynę. Stwierdził, że jeśli maszyna może być częścią rozmowy za pomocą teleprinteru i naśladuje człowieka tak całkowicie, że nie ma zauważalnych różnic, to można uznać, że maszyna jest zdolna do myślenia.
- Wkrótce po tym, w 1952 roku, model Hodgkina-Huxleya pokazał, w jaki sposób mózg wykorzystuje neurony do tworzenia sieci elektrycznej.
- Wydarzenia te pomogły zainspirować ideę sztucznej inteligencji (AI), przetwarzania języka naturalnego (NLP) i ewolucji komputerów.

Elementy historii

- Noam Chomsky opublikował Syntactic Structures w 1957 roku. W tej książce zrewolucjonizował koncepcje lingwistyczne i doszedł do wniosku, że aby komputer mógł zrozumieć język, struktura zdania musiałaby zostać zmieniona. Mając to na uwadze, Chomsky stworzył styl gramatyki zwany gramatyką fazowo-strukturalną, który metodycznie tłumaczył zdania języka naturalnego na format użyteczny dla komputerów. (Ogólnym celem było stworzenie komputera zdolnego do naśladowania ludzkiego mózgu pod względem myślenia i komunikacji - sztucznej inteligencji).

Elementy historii

- W 1958 roku John McCarthy wydał język programowania LISP (Locator/Identifier Separation Protocol), język komputerowy używany do dziś.
- W 1964 roku opracowano **ELIZA**, proces „pisanie na maszynie” komentarzy i odpowiedzi, zaprojektowany w celu naśladowania psychiatri przy użyciu technik refleksji. (Odbywało się to poprzez przestawianie zdań i przestrzeganie stosunkowo prostych zasad gramatycznych, ale komputer nie był w stanie ich zrozumieć). Również w 1964 roku amerykańska Narodowa Rada Badawcza (NRC) utworzyła Komitet Doradczy ds. Automatycznego Przetwarzania Języka (Automatic Language Processing Advisory Committee, w skrócie ALPAC). Zadaniem tego komitetu była ocena postępów w badaniach nad przetwarzaniem języka naturalnego.

Elementy historii

- W 1966 roku NRC i ALPAC zainicjowały pierwsze zatrzymanie AI i NLP, wstrzymując finansowanie badań nad przetwarzaniem języka naturalnego i tłumaczeniem maszynowym.
- Po 12 latach badań i 20 milionach dolarów, **tłumaczenia maszynowe były nadal droższe niż ręczne tłumaczenia ludzkie i wciąż nie było komputerów, które zbliżyłyby się do możliwości prowadzenia podstawowej rozmowy.**
- W 1966 roku badania nad sztuczną inteligencją i przetwarzaniem języka naturalnego (NLP) były przez wielu (choć nie wszystkich) uważane za ślepy zaułek.

Elementy historii

SHRDLU – system komputerowy z dziedziny sztucznej inteligencji służący do przetwarzania języka naturalnego, napisany przez Terry'ego Winograda w MIT Artificial Intelligence Laboratory w latach 1968–1970. SHRDLU umożliwia prostą konwersację (poprzez terminal) z użytkownikiem na temat małego świata obiektów.

Elementy historii

Przetwarzanie języka naturalnego i badania nad sztuczną inteligencją potrzebowały prawie 14 lat (do 1980 r.), aby otrząsnąć się z niespełnionych oczekiwań stworzonych przez ekstremalnych entuzjastów. Pod pewnymi względami zatrzymanie sztucznej inteligencji zapoczątkowało nową fazę świeżych pomysłów, z porzuceniem wcześniejszych koncepcji tłumaczenia maszynowego i nowymi pomysłami promującymi nowe badania, w tym systemy eksperckie. **Mieszanie lingwistyki i statystyki, które było popularne we wczesnych badaniach NLP, zostało zastąpione tematem czystej statystyki.** Lata 80. zapoczątkowały fundamentalną reorientację, w której proste przybliżenia zastąpiły głęboką analizę, a proces oceny stał się bardziej rygorystyczny.

Elementy historii

Do lat 80. większość systemów NLP wykorzystywała złożone, „odręczne” reguły. Pod koniec lat 80. nastąpiła rewolucja w NLP. **Było to wynikiem zarówno stałego wzrostu mocy obliczeniowej, jak i przejścia na algorytmy uczenia maszynowego.** Podczas gdy niektóre z wczesnych algorytmów uczenia maszynowego (drzewa decyzyjne stanowią dobry przykład) stworzyły systemy podobne do odręcznych reguł starej szkoły, badania w coraz większym stopniu koncentrowały się na modelach statystycznych.

W latach 80-tych IBM był odpowiedzialny za opracowanie kilku udanych, skomplikowanych **modeli statystycznych.**

Elementy historii

W latach 90. popularność **modeli statystycznych** do analizy procesów języka naturalnego gwałtownie wzrosła. Czysto statystyczne metody NLP stały się niezwykle cenne w nadążaniu za ogromnym przepływem tekstu online. **N-Gramy stały się użyteczne, rozpoznając i śledząc liczbowo skupiska danych językowych.**

W 1997 roku wprowadzono modele rekurencyjnych sieci neuronowych LSTM (RNN), które w 2007 roku znalazły zastosowanie w przetwarzaniu głosu i tekstu.

Obecnie modele sieci neuronowych są uważane za wiodące w badaniach i rozwoju w zakresie rozumienia tekstu i generowania mowy przez NLP.

N-gram – model językowy stosowany w rozpoznawaniu mowy

Zastosowanie n-gramów wymaga zgromadzenia odpowiednio dużego zasobu danych statystycznych – korpusu. Utworzenie modelu n-gramowego zaczyna się od zliczania wystąpień sekwencji o ustalonej długości n w istniejących zasobach językowych. Zwykle analizuje się całe teksty i zlicza wszystkie pojedyncze wystąpienia (1-gramy, unigramy), dwójki (2-gramy, bigramy) i trójki (3-gramy, trigramy). Aby uzyskać 4-gramy słów potrzebnych jest bardzo dużo danych językowych, co szczególnie dla języka polskiego jest trudne do zrealizowania.

Google Ngram Viewer lub Google Books Ngram Viewer to wyszukiwarka internetowa, która wykreśla częstotliwości dowolnego zestawu wyszukiwanych ciągów znaków na podstawie rocznej liczby n-gramów

Stan obecny

Rok 2023 był rokiem niezwykłego postępu w przetwarzaniu języka naturalnego (NLP) ze względu na udostępnienie GPT3.

Nastąpiła niezwykła ewolucja dużych modeli językowych (LLM), dzięki inicjatywom open source oraz dynamicznej interakcji technologii i etyki w rozwoju sztucznej inteligencji.

Stan obecny

Chatbot Arena

<https://lmarena.ai/?leaderboard>

Model	Mean win rate
GPT-4o (2024-05-13)	0.938
GPT-4o (2024-08-06)	0.928
DeepSeek v3	0.908
Claude 3.5 Sonnet (20240620)	0.885

HELM

<https://crfm.stanford.edu/helm/lite/latest/#/leaderboard>

Rank* (UB)	Rank (StyleCtrl)	Model
1	1	Gemini-2.5-Pro-Exp-03-25
2	1	o3-2025-04-16
2	3	ChatGPT-4o-latest (2025-03-26)
3	5	Grok-3-Preview-02-24
3	5	Gemini-2.5-Flash-Preview-04-17

Open LLM

https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard

Stan obecny

Rok 2023 przyniósł rozkwit nowych metod: jak prawidłowo oceniać LLM (LMSys [https://t.me/rybolos_channel/742], LLM Arena [https://t.me/rybolos_channel/963], Mera [https://t.me/rybolos_channel/955], inne benchmarki [https://t.me/rybolos_channel/705]), zwalczać wycieki danych ([https://t.me/rybolos_channel/963]) i weryfikować rzeczywiste istnienie pojawiających się zdolności ([https://t.me/rybolos_channel/994]).

- Architektury typu Mixture of Expert pojawiły się po raz drugi.
- Architektury LLM są często centralnym elementem systemów multimodalnych (obok przetwarzania obrazu).

Praca z tekstem

1. Przygotowanie danych tekstowych
 - a. Czyszczenie tekstu przy użyciu biblioteki NLTK lub Keras
 - b. Tokenizacja tekstu
2. Model języka Bag-of-Words
 - a. Przygotowanie słownika metodami: CountVectorizer, TfidfVectorizer, HashingVectorizer
 - b. Zarządzanie słownikiem: punktacja, ograniczanie, archiwizacja

3. Analiza sentymentów

- a. Przygotowanie danych tekstowych
- b. Modele prognostyczne Sentiment Analysis
- c. Klasyfikacja tekstu
- d. Porównywanie i ocena modeli

4. Word Embeddings

- a. Osadzanie słów – algorytmy Continuous-bag-of-words i Skip-gram
- b. Wizualizacja osadzania słów: PCA, PHATE
- c. Word2Vec Embedding i Stanford's GloVe Embedding

5. Generowanie tekstu

- a. Modele sieci neuronowych do generowania dokumentów
- b. Statystyczne modelowanie warstwy semantycznej języka
- c. Opracowanie modelu Embedding + LSTM do generowania tekstu

Modele języka w Polsce

<https://clip.ipipan.waw.pl/>

PLLuM



PLLuM

Polish Large Language Model

Logo PLLuM

Autor	Ministerstwo Cyfryzacji
Pierwsze wydanie	24 lutego 2025
Rodzaj	duży model językowy
Licencja	CC-BY-NC-4.0, Apache License 2.0, Llama 3.3

Mistral AI



Porozmawiaj z Bielikiem



Bielik.AI – europejska rodzina otwartych modeli językowych, stworzona w Polsce.

Otwarte, bezpłatne i bezpieczne. Rozmawiaj z Bielikiem, eksperymentuj i buduj własne rozwiązania



Korzystaj z Bielika

Zainstaluj Bielika lokalnie

Polski Bielik obnaża ograniczenia ChatGPT || Remigiusz Kinas - didaskalia#170



didaskalia

419 tys. subskrybentów



4,8 tys.



Udostępnij

Zapisz

192 tys. wyświetleń 2 miesiące temu Didaskalia

Ponowne odtwarzanie czatu na żywo



Bielik to polski otwarty model językowy
stworzony przez zespół SpeakLeash we
współpracy z ACK Cyfronet AGH.

Krzysztof Ociepa – współzałożyciel Azurro
– nadzoruje w zespole SpeakLeash trening i
fine-tuning Bielika i wie o nim dosłownie
wszystko. Dzięki dogłębnej wiedzy na temat
modelu jesteśmy w stanie zaproponować
wykorzystanie Bielika dopasowane do
potrzeb klienta.

Worek słów - Bag of Words

Problem z modelowaniem tekstu polega na tym, że jest on chaotyczny, natomiast techniki takie jak algorytmy uczenia maszynowego preferują dobrze zdefiniowane dane wejściowe i wyjściowe o stałej długości.

Algorytmy uczenia maszynowego nie mogą bezpośrednio pracować z surowym tekstem; tekst należy przekonwertować na liczby.

W szczególności na wektory liczb.

Nazywa się to ekstrakcją lub kodowaniem cech.

Popularną i prostą metodą ekstrakcji cech z danych tekstowych jest tzw. model tekstu typu bag-of-words.

Worek słów - Bag of Words

Nazwa worek słów związana jest z faktem, że wszelkie informacje o kolejności lub strukturze słów w dokumencie są odrzucane. Model uwzględnia jedynie to, czy dane słowa występują w dokumencie, a nie gdzie są w dokumencie. Brak uwzględnienia kontekstu.

Przykłady Python

- TfidfVectorizer TFIDF-term frequency (podsumowuje, jak często dane słowo pojawia się w słowniku), inverse document frequency - zmniejsza skalę słów, które często pojawiają się w dokumentach.
- Hashing
- CountVectorizer

Osadzanie słów (Word embedding)

- <https://code.google.com/archive/p/word2vec/>

Osadzanie słów to rodzaj reprezentacji słów, w której słowa o podobnym znaczeniu mają podobną reprezentację.

Zazwyczaj reprezentacja jest wektorem o wartościach rzeczywistych, który koduje znaczenie słowa w taki sposób, że oczekuje się, że słowa znajdujące się bliżej w przestrzeni wektorowej będą miały podobne znaczenie.

Osadzanie słów (Word embedding)

Oprogramowanie do trenowania i używania osadzania słów obejmuje:

- Word2vec Tomáša Mikolova,
- GloVe Uniwersytetu Stanforda,
- GN-GloVe,
- Flair embeddings,
- i inne

Znana biblioteka GENSIM



Radim Řehůřek: Creator of **Gensim** living in Prague.

What is Gensim?

Gensim is a free open-source Python library for representing documents as semantic vectors, as efficiently (computer-wise) and painlessly (human-wise) as possible.

Metody osadzania słów

Metody osadzania słów uczą się reprezentacji wektorowej o wartości rzeczywistej dla słownictwa o określonym rozmiarze ze słownictwa korpusu tekstu.

Proces uczenia się jest albo połączony z modelem sieci neuronowej w ramach jakiegoś zadania, takiego jak klasyfikacja dokumentów, lub jest procesem nienadzorowanym, wykorzystującym statystyki dokumentów.

Embedding Layer- warstwa osadzania

Word2Vec

Word2Vec to metoda skutecznego uczenia się samodzielnego osadzania słów z korpusu tekstowego.

Została ona opracowana przez Tomasa Mikolova i in. w Google w 2013 r. jako odpowiedź na potrzebę uczynienia osadzania w oparciu o sieć neuronową i od tego czasu stała się standardem w opracowywaniu wstępnie wytrenowanego osadzania słów.

Efficient Estimation of Word Representations in Vector Space

Tomas Mikolov

Google Inc., Mountain View, CA
tmikolov@google.com

Kai Chen

Google Inc., Mountain View, CA
kaichen@google.com

Greg Corrado

Google Inc., Mountain View, CA
gcorrado@google.com

Jeffrey Dean

Google Inc., Mountain View, CA
jeff@google.com

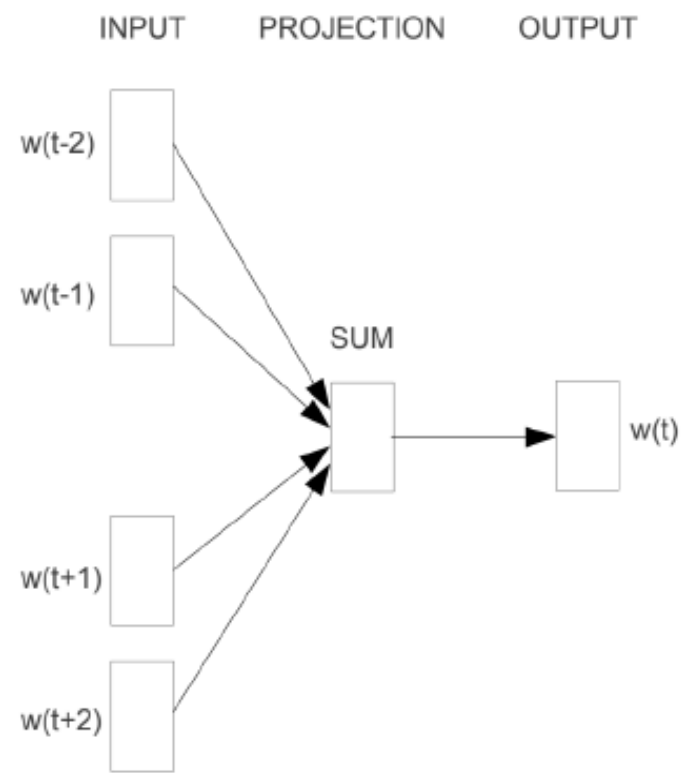
Word2Vec

- Continuous Bag-of-Words lub model CBOW.
- Continuous Skip-Gram Model.

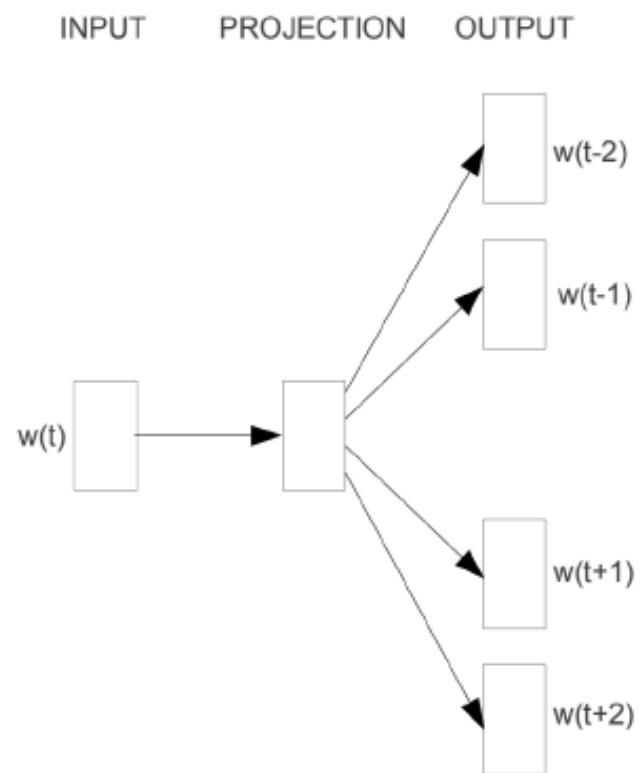
Model CBOW uczy się osadzania, przewidyując bieżące słowo na podstawie jego kontekstu.

Model ciągłego skip-gramu uczy się, przewidyując otaczające słowa na podstawie bieżącego słowa.

<https://www.tensorflow.org/text/tutorials/word2vec>



CBOW

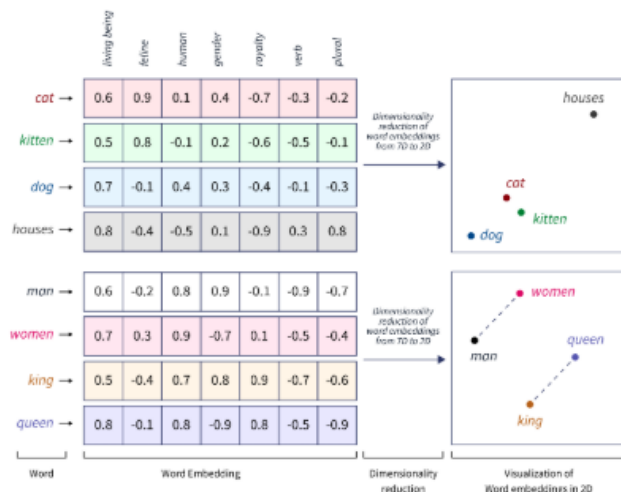


Skip-gram

Osadzanie słów

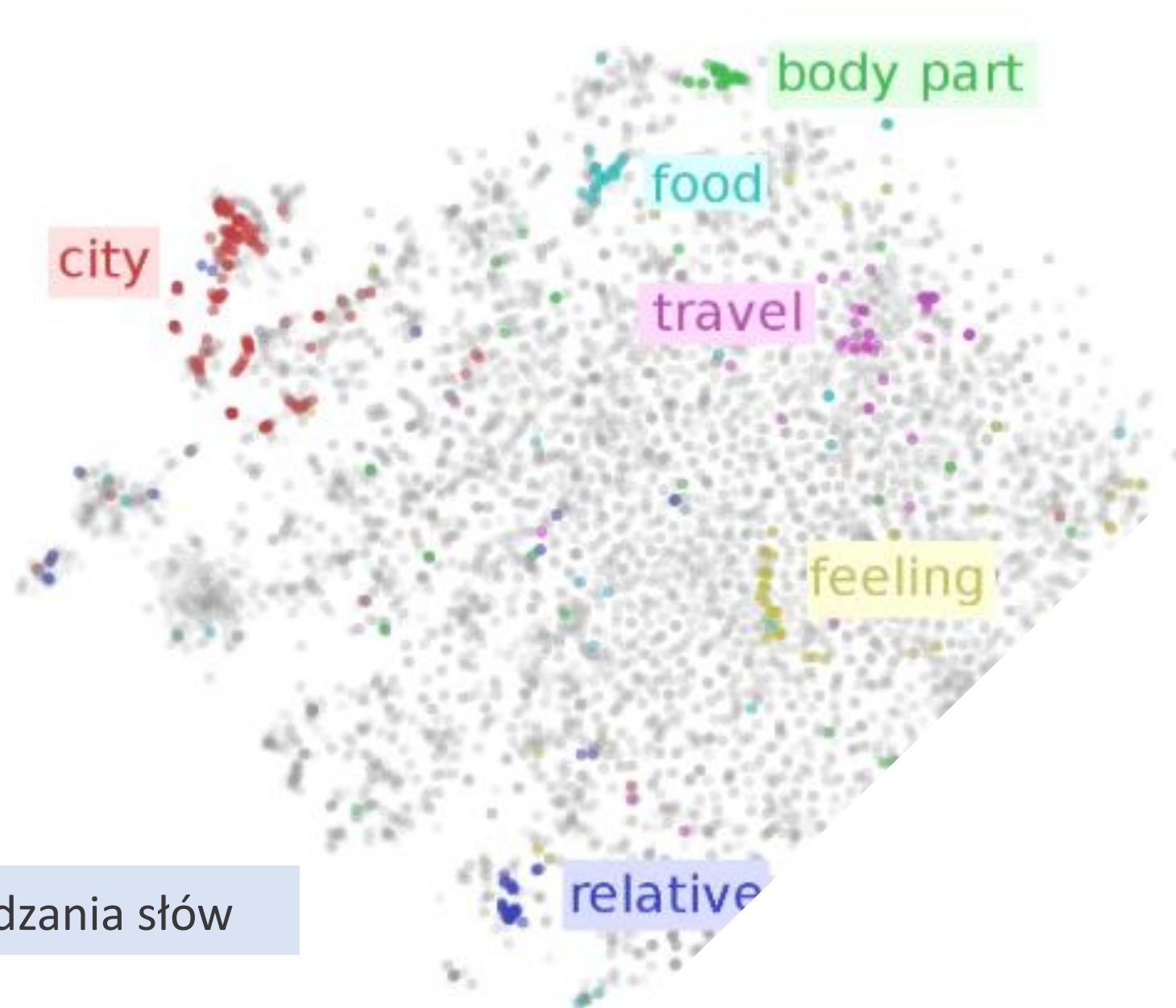
Dane przygotowane jako tokeny (pnie, chunks).

Do tokenów stosujemy osadzanie słów (word embedding)- tokeny reprezentowane są jako wielowymiarowe wektory.



Osadzanie słów to język AI. Każde słowo jest reprezentowane jako wielowymiarowy wektor, który zawiera jego znaczenie semantyczne w oparciu o kontekst w danych uczących. Te wektory pozwalają AI zrozumieć relacje i podobieństwa między słowami, zwiększając zrozumienie i wydajność modelu.

Wizualizacje osadzania słów (PCA, tSNE)



Algorytmy word2vec i GloVe osadzania słów

Przykład

Plik Excel

Architektura Encoder-decoder

Encoder-dekoder (koder-dekoder) to rodzaj architektury sieci neuronowej, która jest używana do uczenia sekwencyjnego. Składa się ona z dwóch części, kodera i dekodera. Koder przetwarza sekwencję wejściową w celu utworzenia zestawu wektorów kontekstowych, które są następnie wykorzystywane przez dekodera do generowania sekwencji wyjściowej. Architektura ta umożliwia między innymi zadania takie jak tłumaczenie maszynowe, streszczanie tekstu, napisy do obrazów. Chodzi o to, aby móc przyjąć jedną formę danych (taką jak tekst) i przekonwertować ją na inną (taką jak obrazy). W ten sposób maszyny mogą nauczyć się rozumieć złożone relacje między różnymi typami danych i wykorzystywać je do bardziej wydajnego przetwarzania.

Architektura Encoder-decoder seq2seq

W podstawowym modelu seq2seq bez uwagi/attention, wektor kontekstu jest zazwyczaj końcowym stanem ukrytym generowanym przez koder (h).

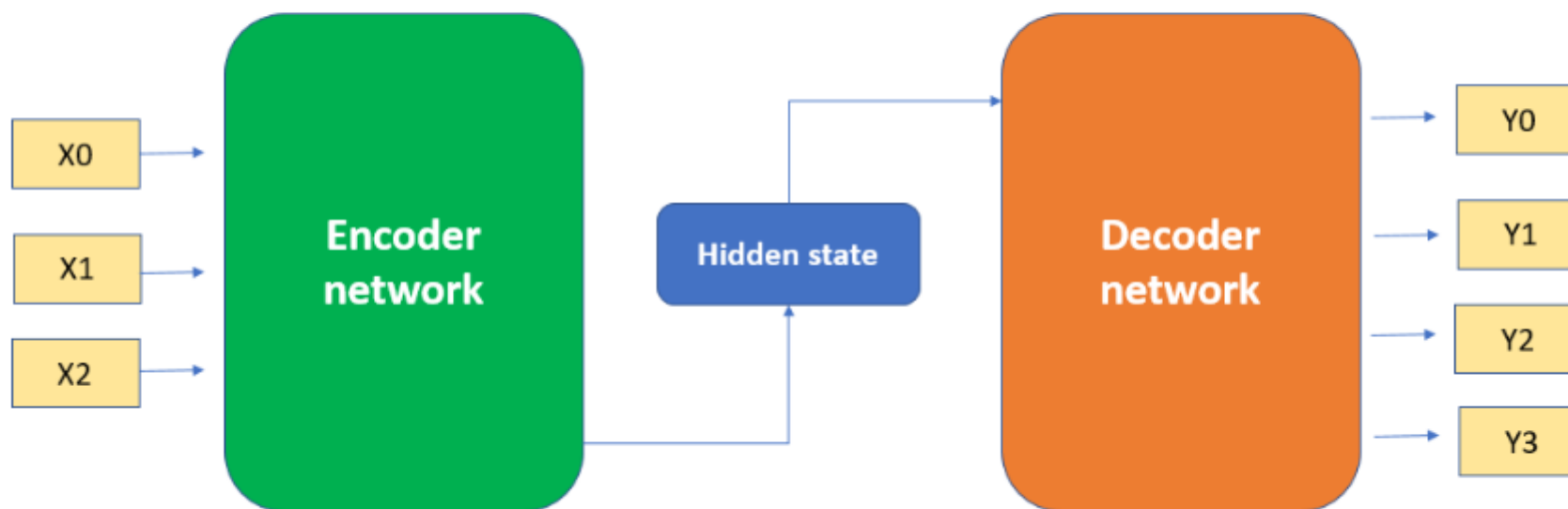
Ten stan ukryty jest następnie używany jako początkowy stan ukryty dla dekodera, który generuje sekwencję wyjściową krok po kroku.

Podczas dekodowania wektor kontekstu jest używany przez dekodera do generowania każdego elementu sekwencji wyjściowej. W każdym kroku dekodera pobiera poprzedni element wyjściowy i bieżący stan ukryty, a następnie tworzy nowy stan ukryty i element wyjściowy

Architektura koder-dekoder z sieciami neuronowymi

- Możemy wykorzystać CNN, RNN i LSTM w architekturze koder-dekoder do rozwiązywania różnego rodzaju problemów. Wykorzystanie kombinacji różnych typów sieci może pomóc w uchwyceniu złożonych relacji pomiędzy sekwencją danych wejściowych i wyjściowych.
- CNN jako koder, RNN/LSTM jako dekodek: Ta architektura może być używana do zadań takich jak napisy do obrazów, gdzie wejściem jest obraz, a wyjściem jest sekwencja słów opisujących obraz. CNN może wyodrębnić cechy z obrazu, podczas gdy RNN/LSTM może wygenerować odpowiednią sekwencję tekstową. CNN są dobre w wyodrębnianiu cech z obrazów i dlatego mogą być używane jako koder w zadaniach związanych z obrazami.
- RNN/LSTM są dobre w przetwarzaniu danych sekwencyjnych, takich jak sekwencje słów, i mogą być używane jako kodery w zadaniach związanych z obrazami.

Przykład Excel



- <https://pytorch.org/text/main/datasets.html>

Datasets

- Text Classification

- AG_NEWS
- AmazonReviewFull
- AmazonReviewPolarity
- CoLA
- DBpedia
- IMDb
- MNLI
- MRPC
- QNLI
- QQP
- RTE
- SogouNews
- SST2
- STSB
- WNLI

torchtext.datasets

- + Text Classification
- + Language Modeling
- + Machine Translation
- + Sequence Tagging
- + Question Answer
- + Unsupervised Learning

- torchtext.nn

- MultiheadAttentionContainer
- InProjContainer
- ScaledDotProduct

- torchtext.data.functional

- generate_sp_model
- load_sp_model
- sentencepiece_numericalizer
- sentencepiece_tokenizer
- custom_replace
- simple_space_split
- numericalize_tokens_from_iterator
- filter_wikipedia_xml
- to_map_style_dataset

- torchtext.data.metrics

- bleu_score

- torchtext.data.utils

- get_tokenizer
- ngrams_iterator

Duże modele językowe LLM

Duże modele językowe, to zaawansowane systemy sztucznej inteligencji zaprojektowane do rozumienia i generowania tekstu podobnego do ludzkiego na podstawie dostarczonych im danych wejściowych. Modele te charakteryzują się ogromną skalą, z miliardami, a nawet bilionami parametrów, które pozwalają im uchwycić skomplikowane wzorce i niuanse w języku.

2017 r.

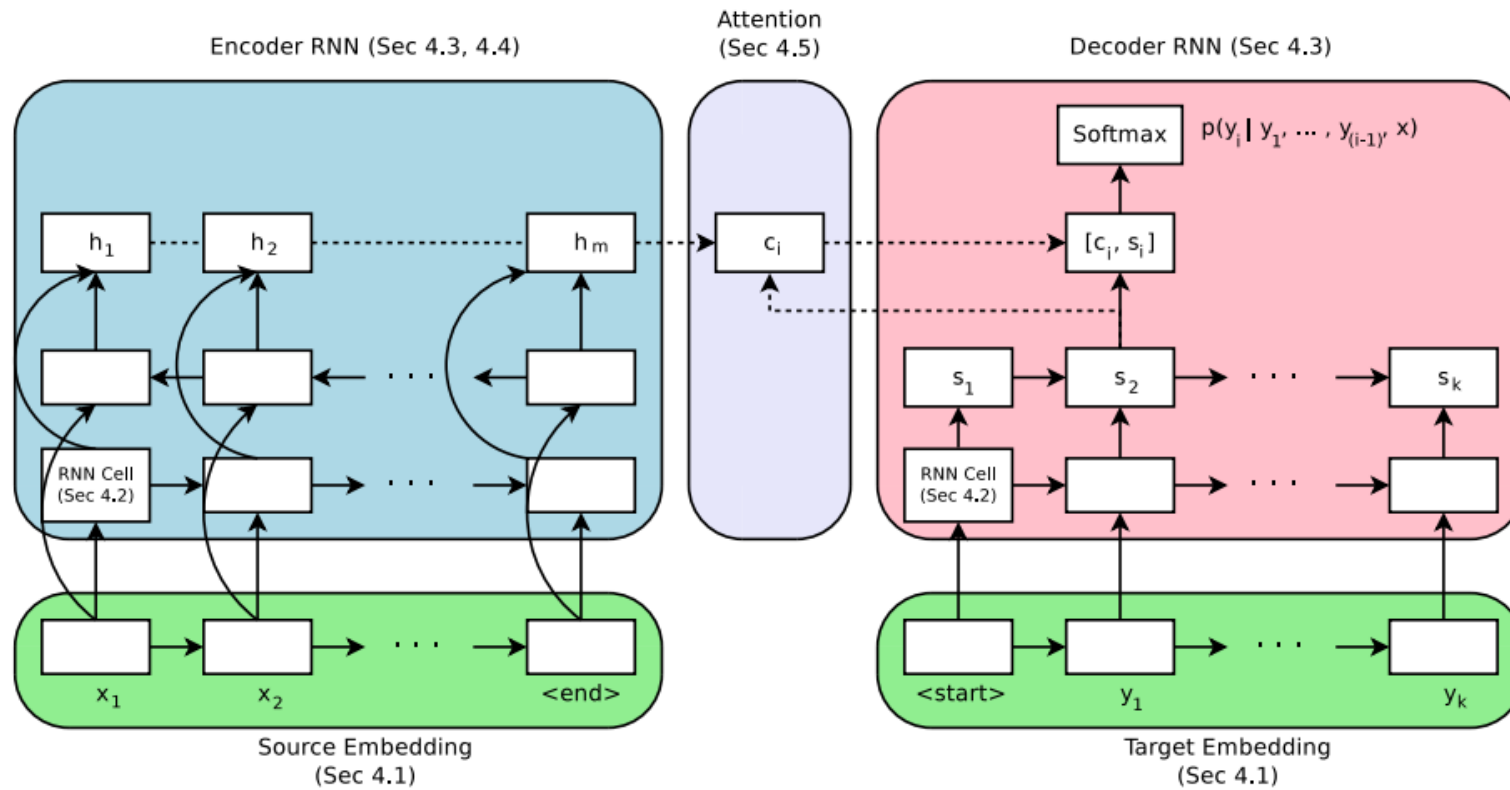


Figure 1: Encoder-Decoder architecture with attention module. Section numbers reference experiments corresponding to the components.

Massive Exploration of Neural Machine Translation Architectures

Denny Britz[†], Anna Goldie^{*}, Minh-Thang Luong, Quoc Le
{dennybritz, agoldie, thangluong, qvl}@google.com
Google Brain

Uwaga/Attention

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Biogram

Łukasz is the co-author of Tensorflow, the Tensor2Tensor and Trax libraries, and the Transformer paper. He is a Staff Research Scientist at Google Brain and his work has greatly influenced the AI community.

Attention is All You Need

A Vaswani · Cytowane przez 185976 –



arXiv

<https://arxiv.org> › cs · [Tłumaczenie strony](#) ⋮

[1706.03762] Attention Is All You Need

12 cze 2017 — We propose a new simple network architecture, the Transformer, based solely on **attention** mechanisms, dispensing with recurrence and convolutions entirely.



Institut Informatyki Uniwersytetu Wrocławskiego
<https://ii.uni.wroc.pl> › [Instytut](#) › [aktualności](#) ⋮

[Łukasz Kaiser - nasz absolwent - wśród autorów systemu ...](#)

18 mar 2023 — Jednym z core contributors systemu był nasz absolwent - Łukasz Kaiser, obecnie pracownik OpenAI, a wcześniej Google Brain. Łukasz pełnił rolę ...

„Japoński Nobel” dla Łukasza Kaisera

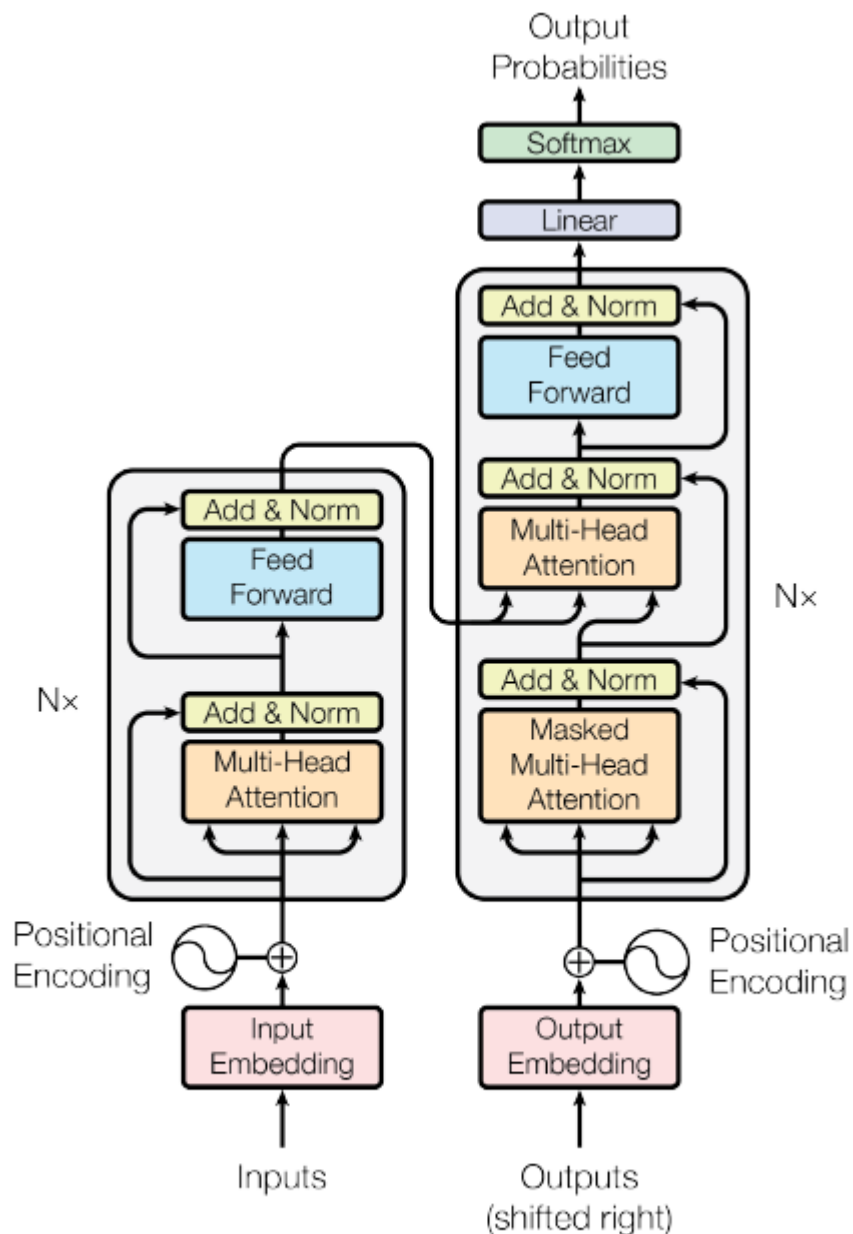
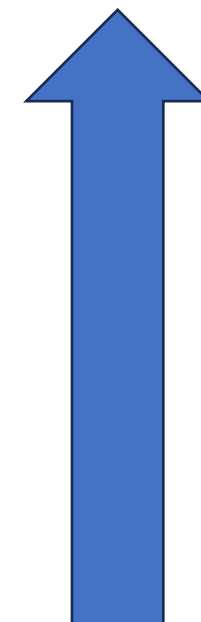


Figure 1: The Transformer - model architecture.

W pracy Attention is ..

Architektura koder- dekodek używana np. do tłumaczenia tekstu.

- Sieć gęsto połączona
- Połączenia resztowe
- Obliczanie samo-uwagi (self-Attention)
- Osadzenie słów i pozycji
- Osadzanie słów



Kodowanie pozycyjne lub pozycyjne osadzanie- wektor E

Zastępuje sekwencyjny wsad RNN (LSTM)

Kodowanie pozycyjne opiera się na wartościach funkcji sin i cos

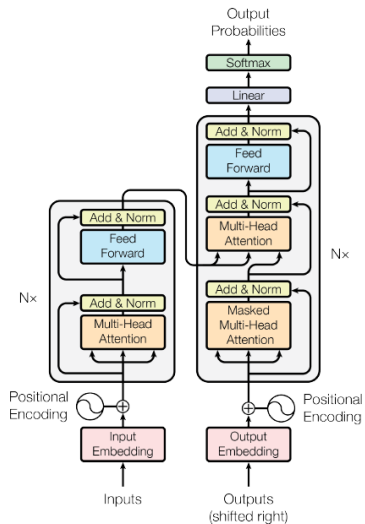
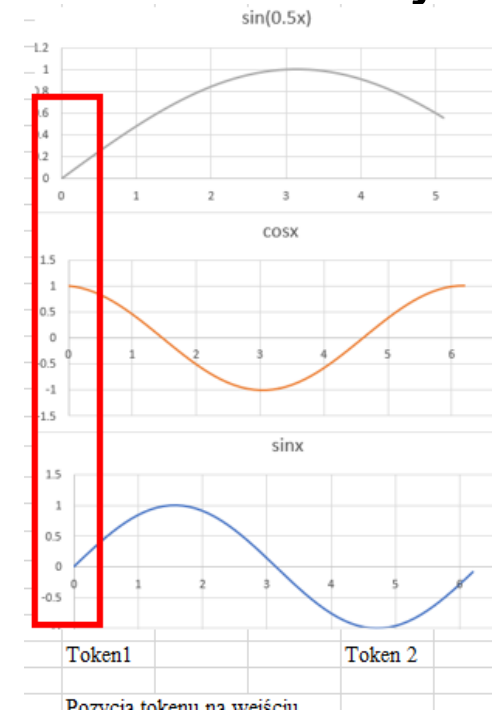


Figure 1: The Transformer - model architecture.

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$



pozycyjne osadzanie-inna technika z pracy Gehring, Auli

1. Osadzanie słów (Word Embedding)

Macierz osadzania W_E : każdemu tokenowi odpowiada wektor o zadanym wymiarze.
GPT3 –wektor o 12 288 współrzędnych.

Macierz osadzania ma wymiar 12 288 wiersz i 50 257 kolumn.

Wagi macierzy osadzania W_E są wyznaczane w procesie uczenia.

```
CLASS torch.nn.Embedding(num_embeddings, embedding_dim, padding_idx=None,  
                        max_norm=None, norm_type=2.0, scale_grad_by_freq=False, sparse=False,  
                        _weight=None, _freeze=False, device=None, dtype=None) \[SOURCE\]
```

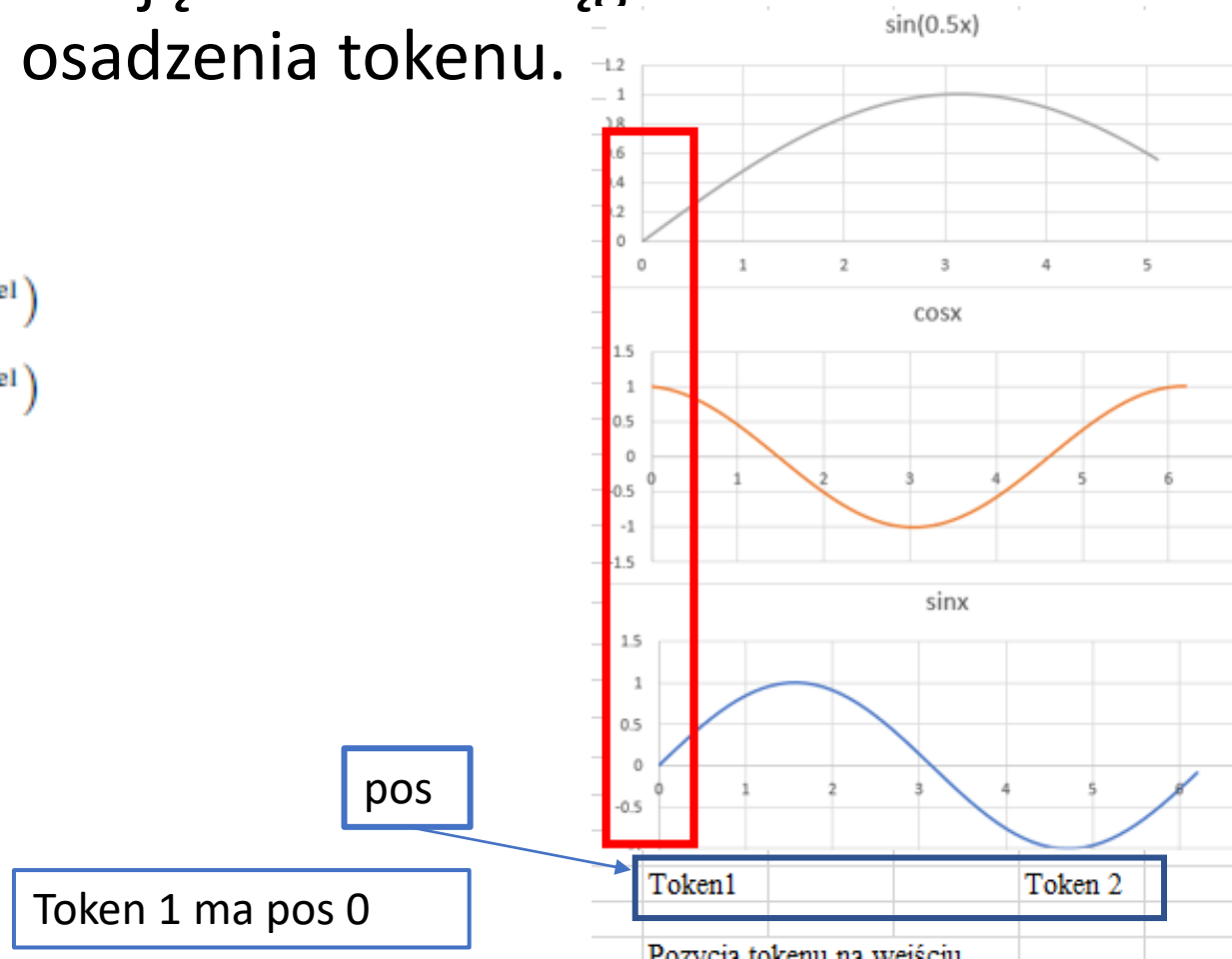
2. Kodowanie pozycyjne

Wartości wektora dla kodowania pozycyjnego mogą się powtarzać, ale daje on jednoznaczną reprezentację tokenu w ciągu tokenów o wymiarze równym wektorowi osadzenia tokenu.

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

d_{model} = wymiar osadzania



3. Obliczanie uwagi

Komórka Samo-Uwagi (Self Attention)

$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

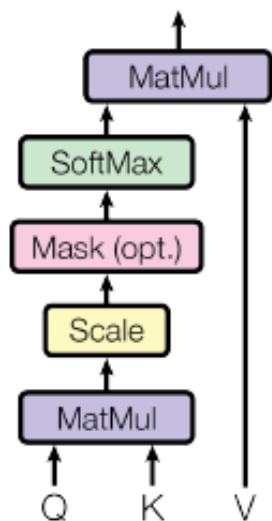
Iloczyn macierzy i osadzeń
poszczególnych tokenów

WQ		E2	Q
-0.8	-1.7	1.45	-2.367
0.4	2.8	0.71	2.568
Wk		E2	K
-1.5	0.7	1.45	-1.678
1.5	-2.1	0.71	0.684
Wk		E1	
-1.5	0.7	-2.38	4.34
1.5	-2.1	1.1	-5.88

Wv		E2	V2
1	-0.5	1.45	1.10
0.6	0.1	0.71	0.94
		E1	V1
1	-0.5	-2.38	-2.93
0.6	0.1	1.1	-1.318

Uwaga/samouwaga

Scaled Dot-Product Attention



- Q-query- zapytanie
- K- key klucz
- V-value wartość

Macierz parametrów W_Q

Macierz parametrów W_K

Macierz parametrów W_V

Wyznaczane w
procesie uczenia

$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Obliczanie uwagi

Komórka Samo-Uwagi (Self Attention)

`torch.nn.functional.cosine_similarity`

`torch.nn.functional.cosine_similarity(x1, x2, dim=1, eps=1e-8) → Tensor`

$$\text{Attention}(Q, K, V) = \text{SoftMax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q-query- zapytanie

K- key -klucz

V-value wartość

d_k -wymiar -wektora zapytania

Kąt między wektorami

Ze wzoru na [iloczyn skalarny](#) można wyprowadzić wzór na cosinus kąta między wektorami.

$$\cos \alpha = \frac{\vec{a} \circ \vec{b}}{|\vec{a}| \cdot |\vec{b}|}$$

$\vec{a} \circ \vec{b}$ – iloczyn skalarny wektorów $\vec{a}$ i $\vec{b}$

$|\vec{a}|, |\vec{b}|$ – długość wektora $\vec{a}$ i $\vec{b}$

$$\text{Cosine Similarity} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Wyznaczanie samouwagi



Samouwaga dla modelu GPT3 wyznaczana jest tylko dla tokenów poprzedzających

Self-Attention (samo-uwaga)

Terminy samo-uwaga i skalowana uwaga iloczynu skalarnego są używane zamiennie.

- Samo-uwaga mierzy jak bardzo słowo powinno skupiać się na innych słowach w zdaniu.

Aby to obliczyć wykonuje się następujące kroki.

1. Tworzy się 3 nowe wektory z każdego wektora słów wejściowych, zwane wektorami **zapytania, klucza i wartości**.

- Wektory te są oznaczone symbolami Q, K i V.
- Są one uzyskane przez pomnożenie wektora słów wejściowych przez trzy zestawy wag znajdujących podczas uczenia i tych samych dla całego promptu.

2. Używamy wektorów zapytań i kluczy do obliczenia wyniku. Wynik ten określa, ile uwagi należy zwrócić na inne słowa w zdaniu dla danego słowa.

Self-Attention (samo-uwaga)

3. Skalowanie wyniku przez stałą, tak aby jego wartość była pod kontrolą (stabilizuje szkolenie).
4. Normalizacja wyniku przy użyciu funkcji softmax, tak aby każdy wynik był dodatni, a suma wyników wszystkich słów w zdaniu wynosiła 1.
5. Pomnożenie wyniku przez wektor wartości, aby obliczyć wartość uwagi.

- Zapytanie reprezentuje bieżące słowo.
- Klucz reprezentuje wszystkie słowa w zdaniu.
- Wartość reprezentuje treść każdego słowa.

4. Maskowanie

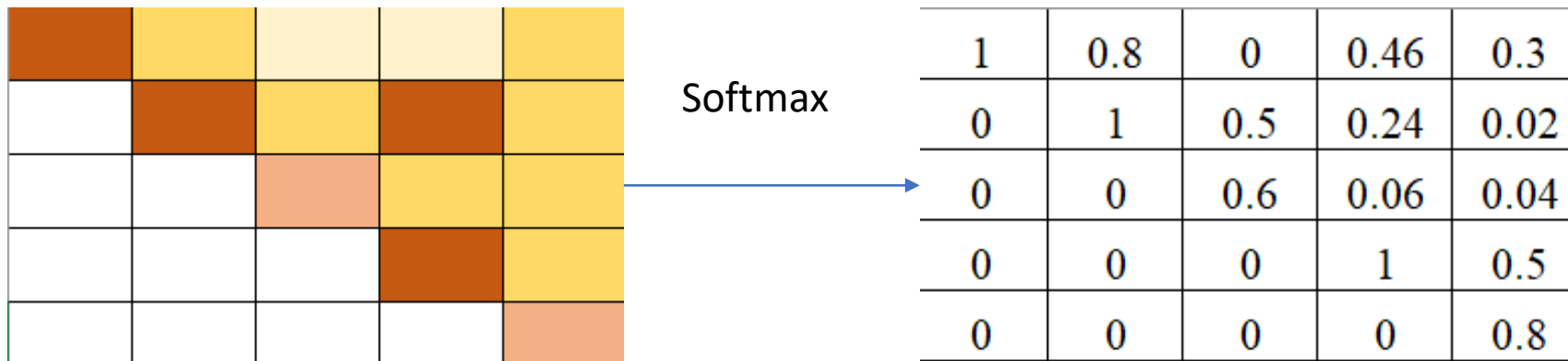
$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

				Token1	Token2	Token3	Token4	Token5
				E1	E2	E3	E4	E5
				E1WQ	E2 WQ	E3 WQ	E4 WQ	E5 WQ
				Q1	Q2	Q3	Q4	Q5
Token1	E1	E1 WK	K1	K1Q1	K1Q2	K1Q3	K1Q4	K1Q5
Token2	E2	E2 WK	K2	K2Q1	K2Q2	K2Q3	K2Q4	K2Q5
Token3	E3	E3 WK	K3	K3Q1	K3Q2	K3Q3	K3Q4	K3Q5
Token4	E4	E4 WK	K4	K4Q1	K4Q2	K4Q3	K4Q4	K4Q5
Token5	E5	E5 WK	K5	K5Q1	K5Q2	K5Q3	K5Q4	K5Q5
Token6	E6	E6 WK	K6	K6Q1	K6Q2	K6Q3	K6Q4	K6Q5
Token7	E7	E7 WK	K7	K7Q1	K7Q2	K7Q3	K7Q4	K7Q5
Token8	E8	E8 WK	K8	K8Q1	K8Q2	K8Q3	K8Q4	K8Q5
Token9	E9	E9 WK	K9	K9O1	K9O2	K9O3	K9O4	K9O5

4. Maskowanie

$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Podczas obliczania wyniku samouwagi maskowanie jest stosowane do licznika tuż przed Softmax.
- Zamaskowane elementy (białe kwadraty) są ustawione na ujemną nieskończoność, więc Softmax zamienia te wartości na zero.



Znormalizowana macierz samouwagi

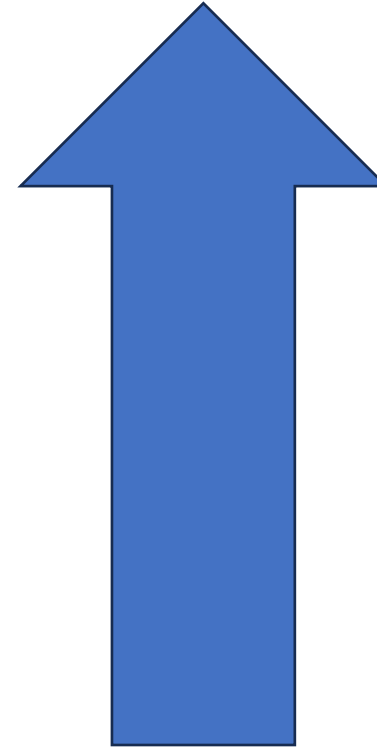
5. Obliczenie połączeń resztowych

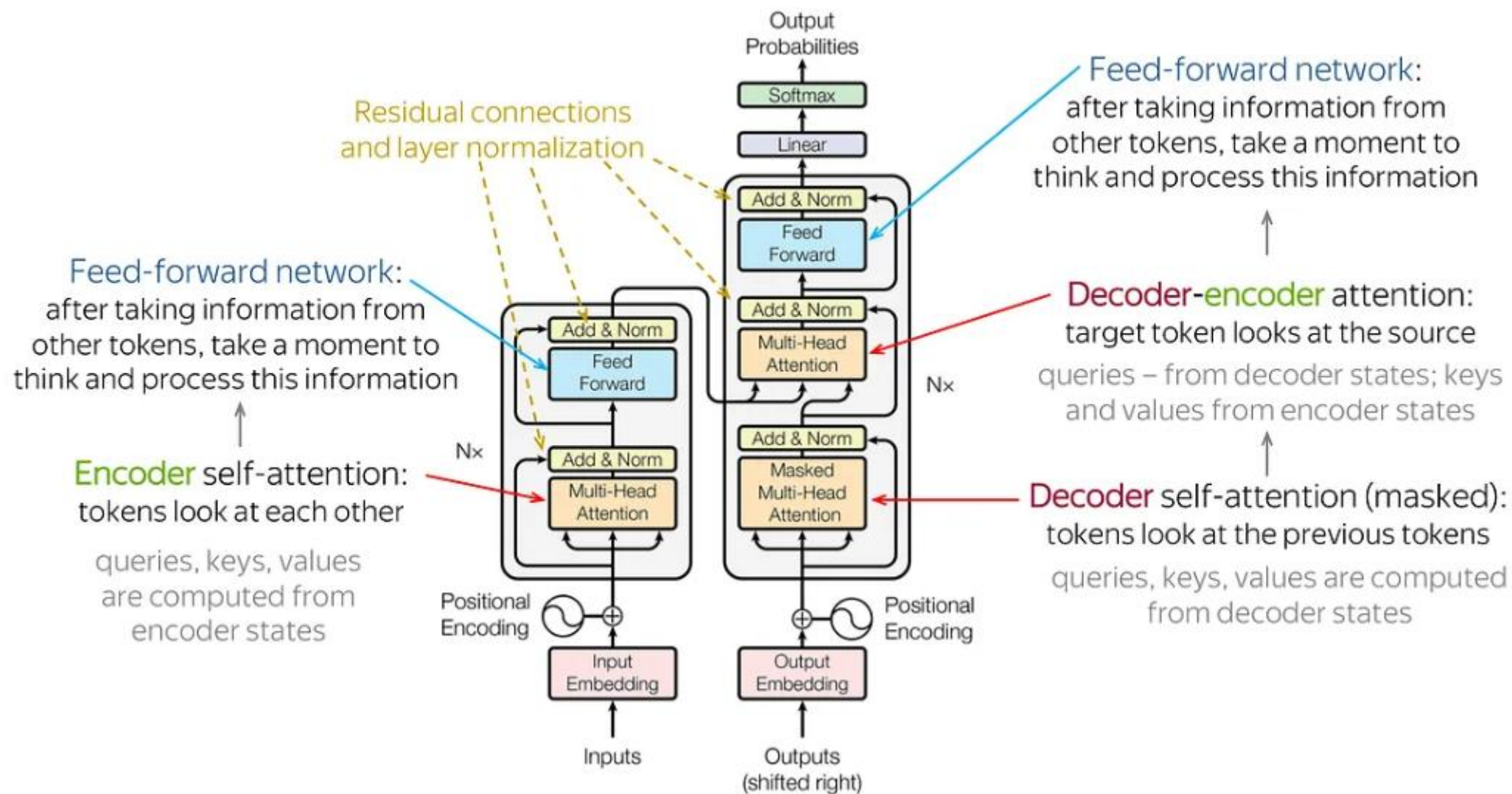
6. Przejęcie przez FFN

7. Wykorzystanie funkcji Softmax do predykcji następnego tokenu.

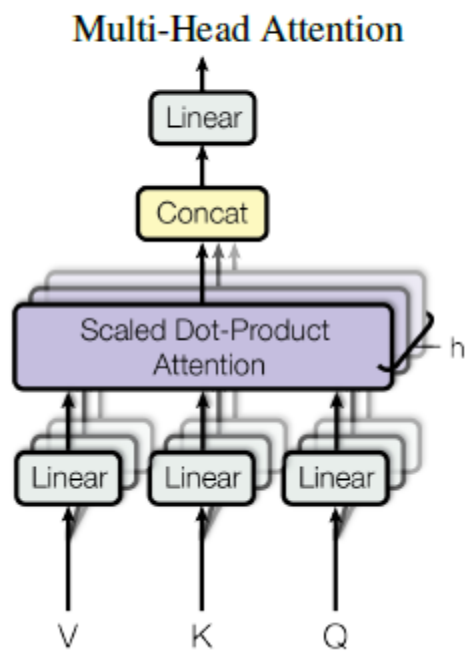
Schemat

- Sieć gęsto połączona
- Połączenia resztowe
- Obliczanie samo-uwagi (self-Attention)
- Osadzenie słów i pozycji
- Osadzanie słów (Word embedding)





[Image Source: [Cross Validated](#)]



```
CLASS torch.nn.Transformer(d_model=512, nhead=8, num_encoder_layers=6,  
num_decoder_layers=6, dim_feedforward=2048, dropout=0.1,  
activation=<function relu>, custom_encoder=None,  
custom_decoder=None, layer_norm_eps=1e-05, batch_first=False,  
norm_first=False, bias=True, device=None, dtype=None) \[SOURCE\]
```

`nn.MultiheadAttention`

`scaled_dot_product_attention`

torch.nn.attention

Elementy historii



GPT- Generative Pre-trained Transformer

Pierwotny model GPT OpenAI („GPT-1”) 2018 r

GPT-2 - 2019 nienadzorowany model języka

Model językowy GPT-3, 2020 r.

Model językowy GPT-3.5

Uruchomienie ChatGPT3- listopad/grudzień 2022 r. –
popularyzacja Transformersów i LLM

DALL-E, 2021, model głębokiego uczenia, który może
generować cyfrowe obrazy z opisów w języku naturalnym

BERT

Bidirectional Encoder Representations from Transformers (BERT) to model językowy oparty na architekturze transformersów, wyróżniający się znaczną poprawą w stosunku do poprzednich modeli.

Został on wprowadzony w październiku 2018 r. przez naukowców z Google.

2015–2018 OpenAI

W grudniu 2015, w San Francisco, ogłoszono założenie OpenAI non-profit. Grupę inwestorów – założycieli stanowili: Sam Altman, Greg Brockman, Reid Hoffman, Jessica Livingston, Peter Thiel, Elon Musk, Amazon Web Services (AWS), Infosys i YC Research, którzy zadeklarowali wpłatę łącznej kwoty 1 miliarda dolarów amerykańskich na założenie organizacji.

Do inwestorów dołączyła grupa dziewięciu światowej klasy naukowców – założycieli: Ilya Sutskever, Greg Brockman, Trevor Blackwell, Vicki Cheung, Andrej Karpathy, Durk Kingma, John Schulman, Pamela Vagata i Wojciech Zaremba.

W grupie doradców znaleźli się: Pieter Abbeel, Yoshua Bengio, Alan Kay, Sergey Levine i Vishal Sikka. Na czele grupy kreatorów przedsięwzięcia stanęli Sam Altman i Elon Musk.

Elementy historii

GPT-3: Its Nature, Scope, Limits, and Consequences

Luciano Floridi^{1,2} · Massimo Chiriatti³

Published online: 1 November 2020

© The Author(s) 2020

<https://newsletter.artofsaience.com/p/vision-transformers-from-idea-to>
<https://jalammar.github.io/illustrated-transformer/>

Architektura GPT3

Model oparty na dekodерze

Zastosowane maskowanie

175 miliardów parametrów dokładnie 175 181 291 520

27 938 macierzy

Wektory osadzenia: wymiar 12 288

(każdy wektor osadzenia (embedding) ma 12 288 współrzędnych)

Wektory Key i Query mają 128 współrzędnych

Kontekst: 2048 tokenów (context size)-jednorazowo przez sieć przechodzi 2048 tokenów.

Ostatnia warstwa-output 50 257 słów (wymiar słownika)

<https://www.youtube.com/watch?v=wjZofJX0v4M> 3Blue1Brown

Transformers (how LLMs work) explained visually | DL5



3Blue1Brown ✓
7,23 mln subskrybentów



Subskrybujesz ▾

Macierze W_Q , W_V i W_K mają 128 wierszy i 12 288 kolumn

GPT-3		Total weights:
		175,181,291,520
Embedding	$\overset{12,288}{d_embed} * \overset{50,257}{n_vocab}$	= 617,558,016
Key	$\overset{128}{d_query} * \overset{12,288}{d_embed} * \overset{96}{n_heads} * \overset{96}{n_layers}$	= 14,495,514,624
Query	$\overset{128}{d_query} * \overset{12,288}{d_embed} * \overset{96}{n_heads} * \overset{96}{n_layers}$	= 14,495,514,624
Value	$\overset{128}{d_value} * \overset{12,288}{d_embed} * \overset{96}{n_heads} * \overset{96}{n_layers}$	= 14,495,514,624
Output	$\overset{12,288}{d_embed} * \overset{128}{d_value} * \overset{96}{n_heads} * \overset{96}{n_layers}$	= 14,495,514,624
Up-projection	$\overset{49,152}{n_neurons} * \overset{12,288}{d_embed} * \overset{96}{n_layers}$	= 57,982,058,496
Down-projection	$\overset{12,288}{d_embed} * \overset{49,152}{n_neurons} * \overset{96}{n_layers}$	= 57,982,058,496
Unembedding	$\overset{50,257}{n_vocab} * \overset{12,288}{d_embed}$	= 617,558,016

Transformer models: an introduction and catalog

Xavier Amatriain xavier@amatriain.net

Ananth Sankar ansankar@linkedin.com

Jie Bing jbing@linkedin.com

Praveen Kumar Bodigutla pbodigutla@linkedin.com

Timothy J. Hazen thazen@linkedin.com

Michael Kazi mkazi@linkedin.com

April 2, 2024

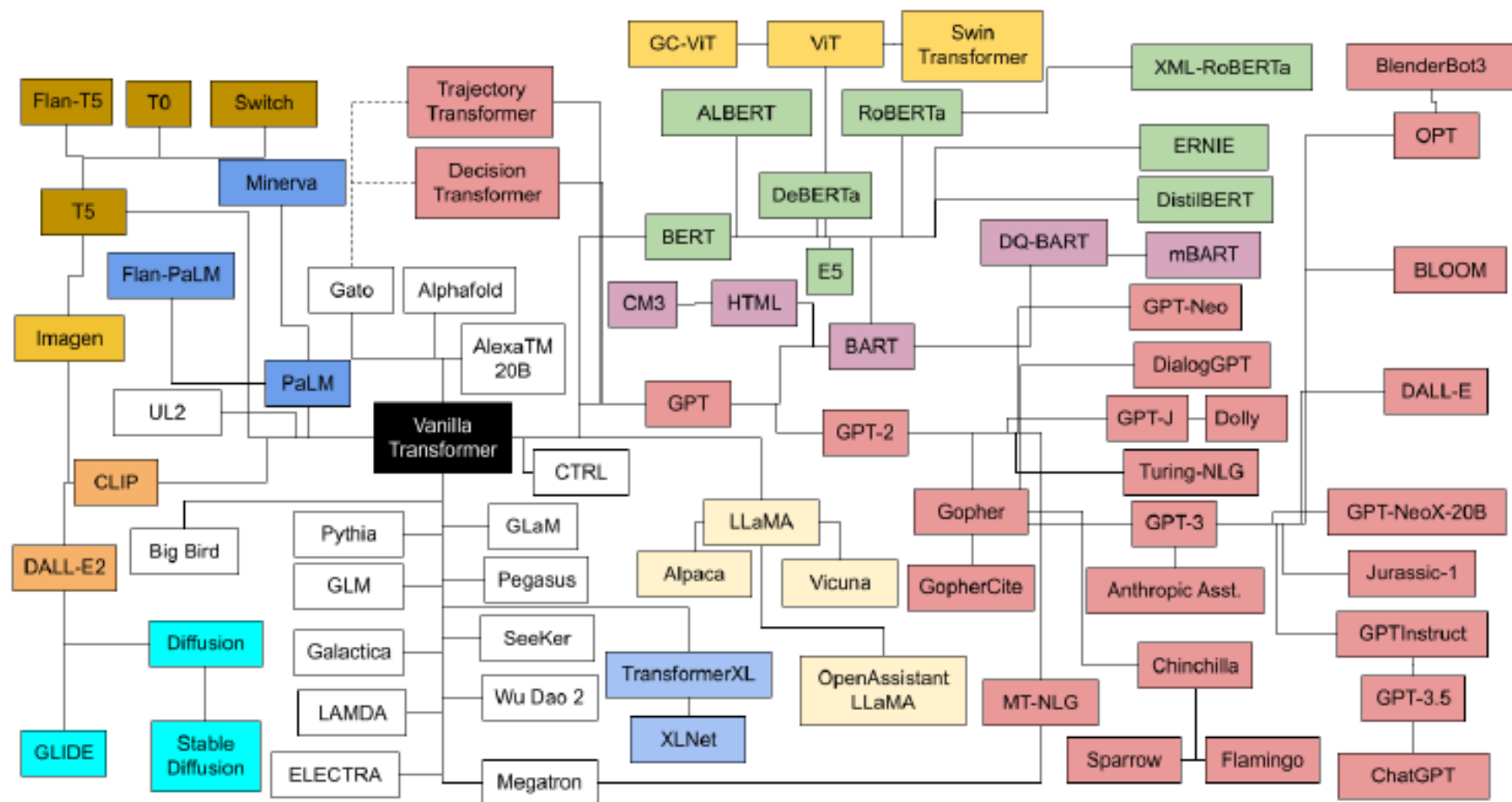


Figure 5: Transformers Family Tree

Źródło: Transformer models: an introduction and catalog

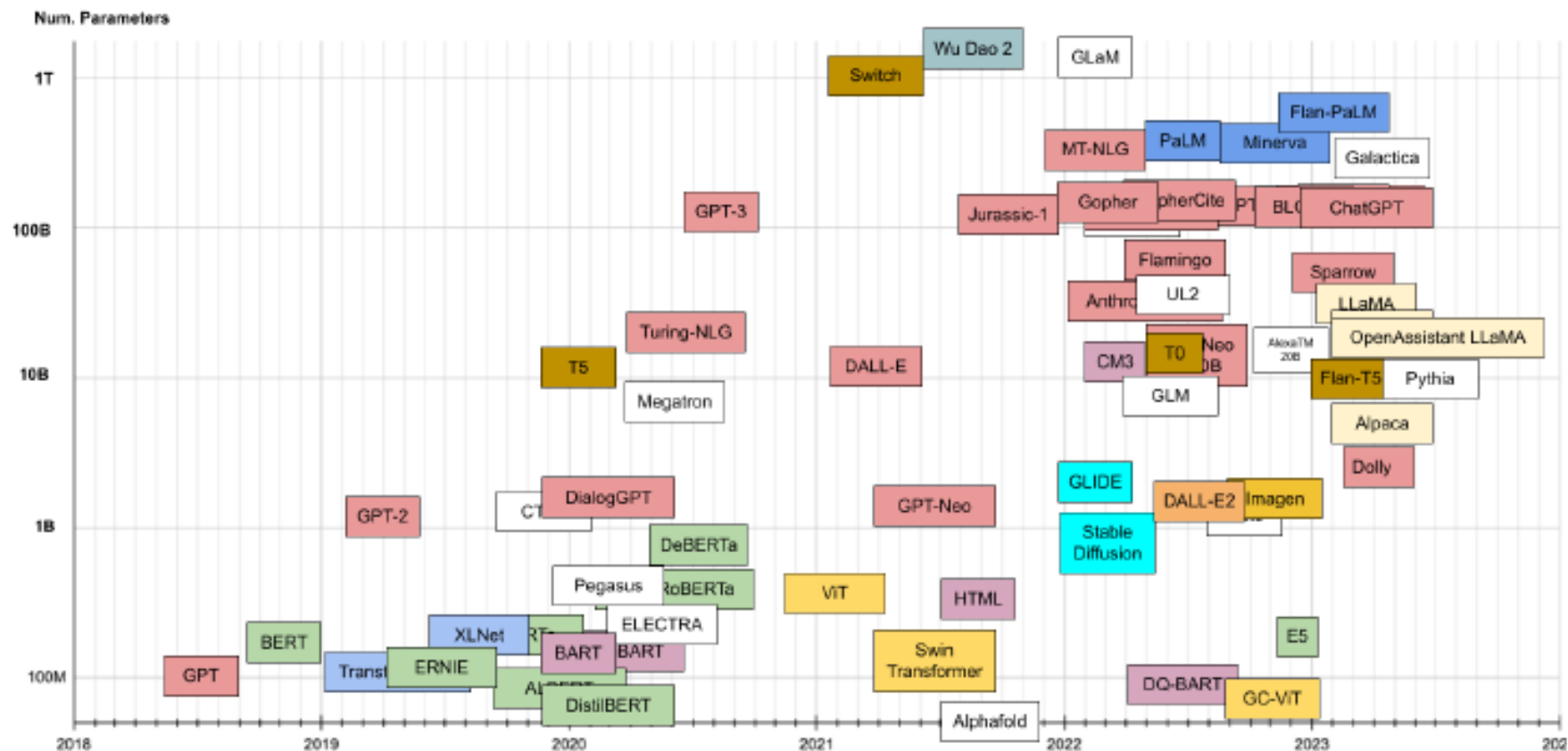


Figure 7: Transformer timeline. On the vertical axis, number of parameters. Colors describe Transformer family.

Źródło: Transformer models: an introduction and catalog

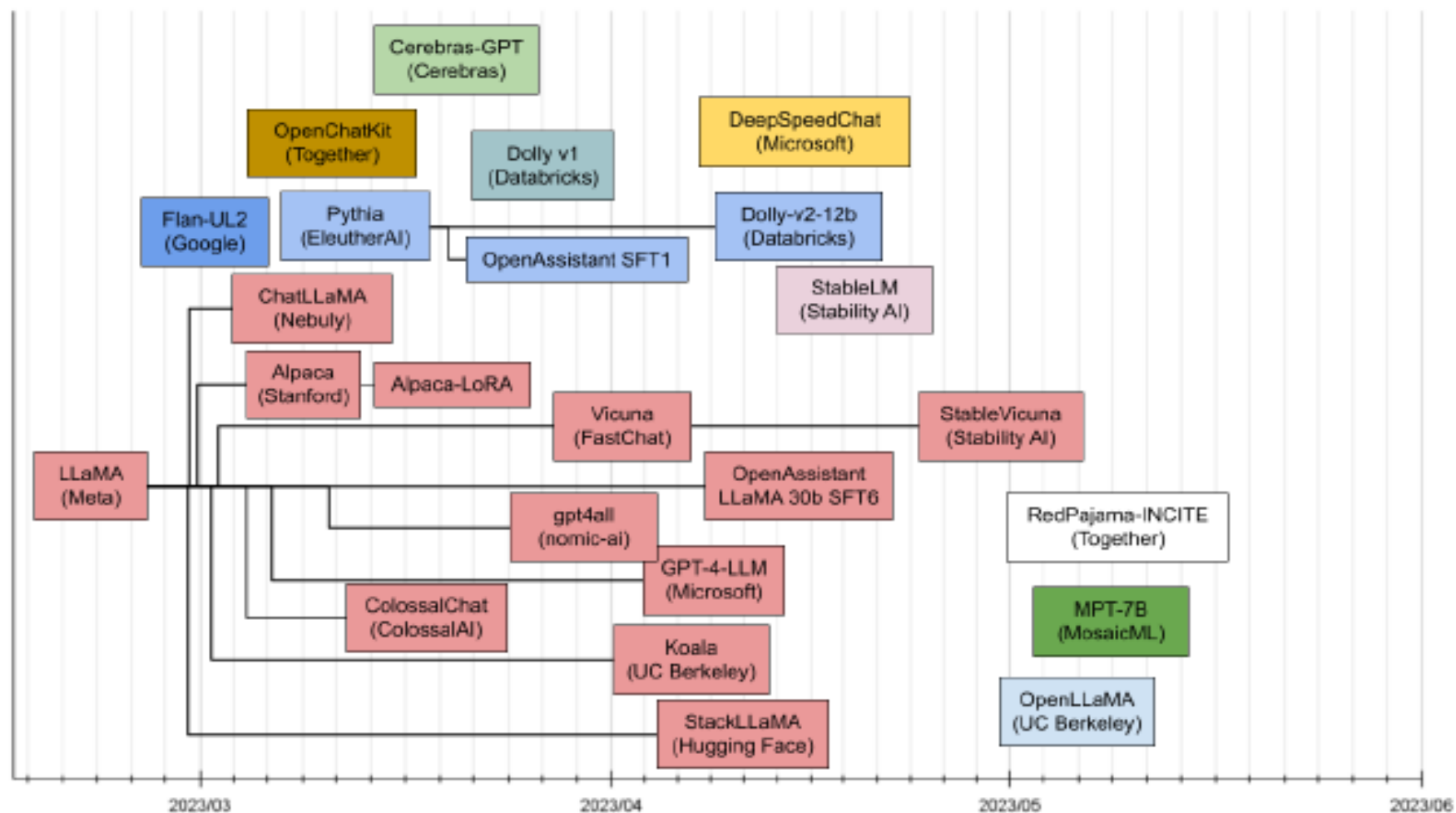
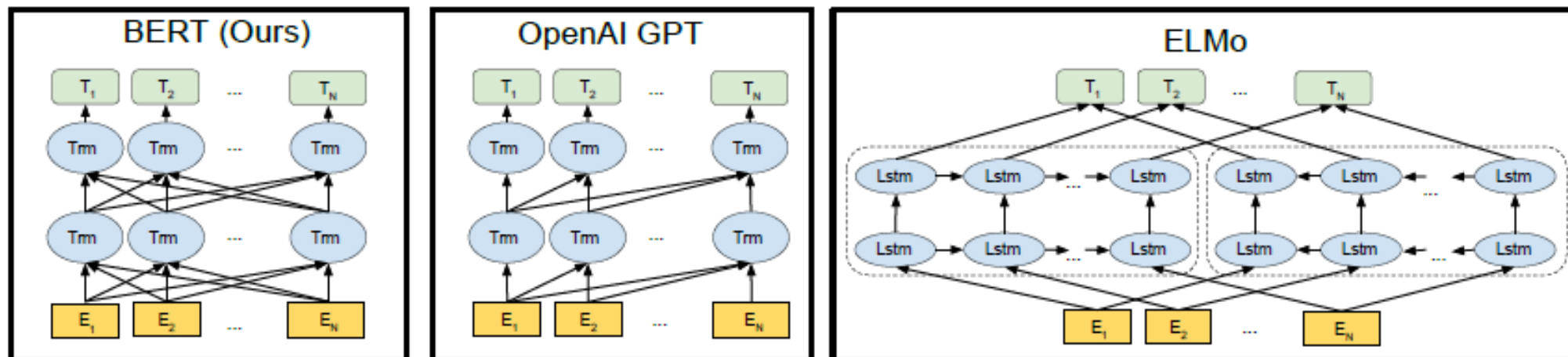


Figure 8: Recently published LLMs

Źródło: Transformer models: an introduction and catalog



Różnice w architekturze modeli pretrained.

- BERT wykorzystuje dwukierunkowy transformer.
- OpenAI GPT wykorzystuje transformer od lewej do prawej.
- ELMo wykorzystuje połączenie niezależnie wytrenowanych modeli LSTM od lewej do prawej i od prawej do lewej do generowania funkcji dla dalszych zadań.

Spośród tych trzech, tylko reprezentacje BERT są wspólnie warunkowane zarówno lewym, jak i prawym kontekstem we wszystkich warstwach.

Źródło: BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Miara dobroci dopasowania modeli NLP

Na przestrzeni lat opracowano wiele różnych wskaźników NLP.

Jednym z najbardziej popularnych jest Bleu Score.

Wyniki Bleu mieszczą się w przedziale od 0 do 1.

Wynik 0,6 lub 0,7 jest uważany za najlepszy z możliwych.

<https://towardsdatascience.com/foundations-of-nlp-explained-bleu-score-and-wer-metrics-1a5ba06d812b>

Miara BLEU

BLEU Score został zaproponowany przez Kishore Papineni i in. w ich artykule z 2002 r. pt. Method for Automatic Evaluation of Machine Translation.

Podejście to polega na zliczaniu dopasowanych n-gramów w tłumaczeniu do n-gramów w tekście referencyjnym, gdzie 1-gram lub unigram to każdy token, a porównanie bigramów to każda para słów. Porównanie jest dokonywane niezależnie od kolejności słów.

Biblioteka Python **Natural Language Toolkit** (NLTK) zapewnia implementację miary BLEU, której można użyć do oceny wygenerowanego tekstu względem odniesienia.

Wskaźnik BLEU dla zdania:

```
from nltk.translate.bleu_score import sentence_bleu
reference = [['this', 'is', 'a', 'test'], ['this', 'is', 'test']]
candidate = ['this', 'is', 'a', 'test']
score = sentence_bleu(reference, candidate)
print(score)
```

```
[6]: ## https://github.com/sdadas/polish-nlp-resources?tab=readme-ov-file#word2vec
##!pip install transformers
from transformers import pipeline
```

```
generator = pipeline("text-generation", model="sdadas/polish-gpt2-medium")
results = generator(seed_text1,
                    max_new_tokens=50, do_sample=True, repetition_penalty = 1.2,
                    num_return_sequences=1, num_beams=1, temperature=0.95, top_k=50, top_p=0.95
                    )
```

Setting `pad\_token\_id` to `eos\_token\_id`:2 for open-end generation.



Sławomir Dadas

National Information Processing Institute, Warsaw, Poland
Zweryfikowany adres z opi.org.pl

machine learning natural language processing

Text oryginalny:

na stronę liberalizmu były dwa odcienie liberalizmu umiarkowany i radykalny chcący przeprowadzić zasadniczą reformę konstytucji do tego radykalnego kierunku przyłączył się bentham w r sam zredagował plan reformy przystąpienie filozofów związanie utylitaryzmu z radykalizmem miało swe uzasadnienie w naczelnej idei utylitaryzmu zgodności interesów w społeczeństwie zgodność interesów może być naturalna jak

Text starter:

na stronę liberalizmu były dwa odcienie liberalizmu umiarkowany i radykalny chcący przeprowadzić zasadniczą reformę konstytucji do tego radykalnego kierunku przyłączył się bentham w r sam zredagował pla

<https://huggingface.co/sdadas>

Text wygenerowany na podstawie startera:

r sam zredagował placet (i to wbrew woli większości swoich wyborców) a na koniec dodał taki jeden z niewielu projektów. I co zrobili te 4 partie? Wszystkie jak pojebane wyklóciły się z nim o jego program gospodarczy, tylko że w żaden sposób nie chcieli go

AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

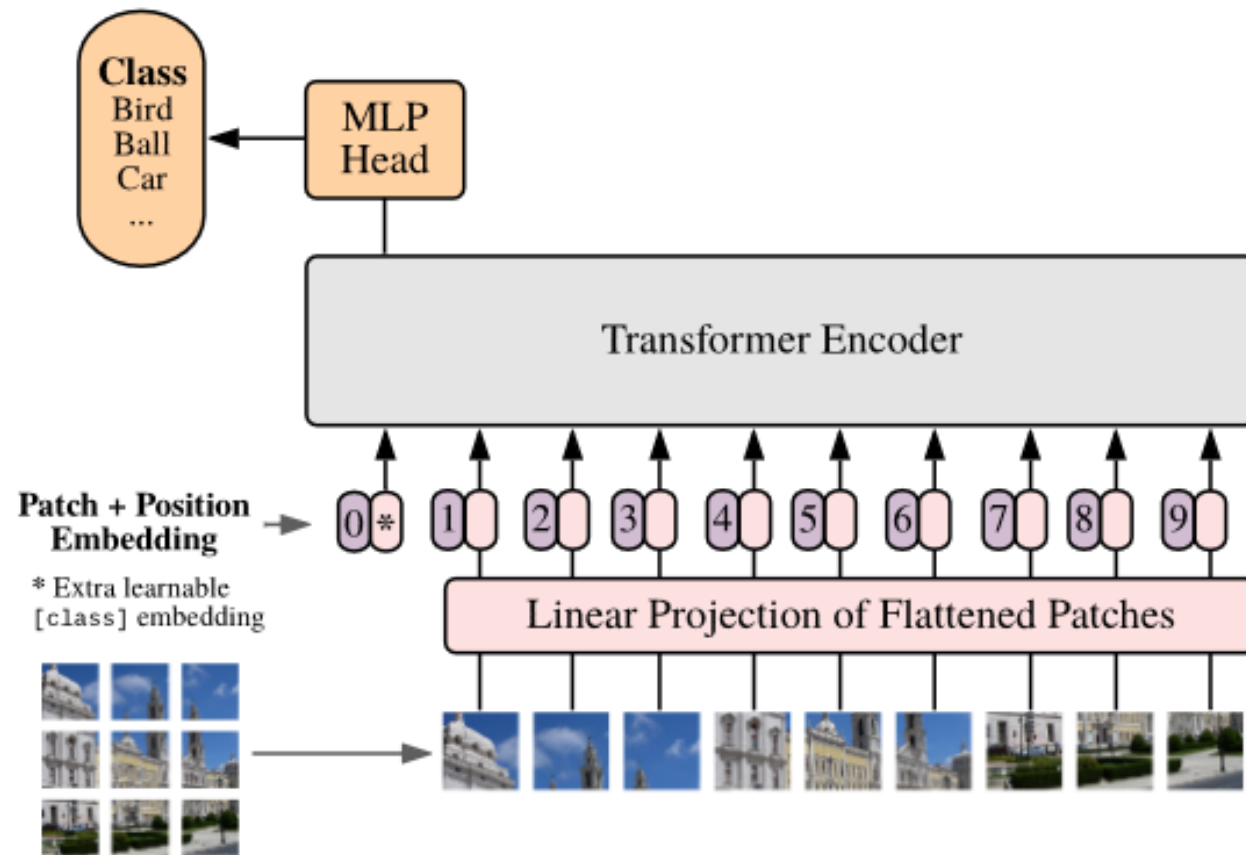
**Alexey Dosovitskiy^{*,†}, Lucas Beyer^{*}, Alexander Kolesnikov^{*}, Dirk Weissenborn^{*},
Xiaohua Zhai^{*}, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby^{*,†}**

^{*}equal technical contribution, [†]equal advising

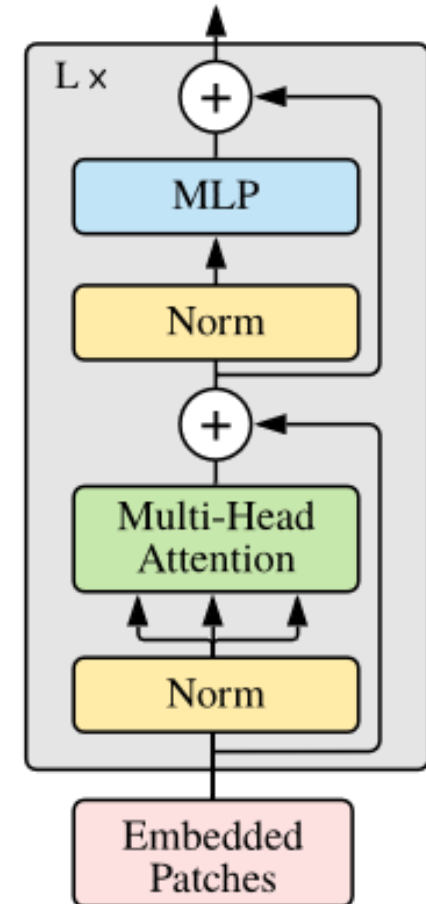
Google Research, Brain Team

`{adosovitskiy, neilhoulby}@google.com`

Vision Transformer (ViT)



Transformer Encoder



How DeepSeek Rewrote the Transformer [MLA]

- https://www.youtube.com/watch?v=0VLAoVGf_74&t=44s